

Let's start by examining the smallest values of n , as they turn out to be special cases:

- The number $n = 1$ is just the digit 1 in every base, so we output the smallest valid base $b = 3$.
- The number $n = 2$ would be 10 in binary, but in every base larger than 2 it is represented by the digit 2. Hence, for $n = 2$ there is no valid base and we output -1 .
- The number $n = 3$ is 10 in ternary, so the smallest possible base is already the correct answer.

Now we can start with the general case. For any $n \geq 4$ we have an easy upper bound on the answer: every such number n is 10 when written in base- n . In fact, we can go one step lower: every such n is 11 when written in base- $(n - 1)$. Thus, some valid bases always exists, we just have to find the smallest one among them.

First subproblem

For the first subtask we can just try converting n into each base from 3 to n , checking whether it contains only 1s and 0s, and output the first one that works. This solution runs in $O(n \log n)$.

Second subproblem

For the second subtask, we have to be a bit more clever about which bases we try. Notice that the last digit of a number in base b is its remainder modulo b ; so if we want the last digit to be 0, the number must be divisible by b (i.e., $n = kb$ for some k), and if we want it to be 1, the number n must give a remainder of one modulo b (i.e., $n = kb + 1$ for some k ; rearranging, $n - 1 = kb$, so $n - 1$ is divisible by b).

Thus a base b has any chance of writing n with 1s and 0s only if it's a divisor of n or $n - 1$. In other words, we just found a necessary condition that will allow us to eliminate most of the bases and speed up the search for one that works.

We can find all divisors of the numbers n and $n - 1$ in $O(\sqrt{n})$ and check them from smallest to largest. This gives us the time complexity of $O(\sqrt{n} \log n)$.

Full solution and beyond

For the final subtask we will make use of the fact that as the base you use gets larger, the number of digits you need to use to represent n gets smaller.

Pick some constant k , and check every base from 3 to k . If you find one that works, that's the answer and we are done.

For any base larger than k , the number n written in base- k will have at most $\log_k n$ digits. And we know that we want each of them to be 0 or 1. We can now try to look for the solution from the opposite end. We'll iterate over every possible binary string of length 1 to $\log_k n$, and for each one we check whether we can find a base b in which the value of this particular string is exactly n .

The easiest way to do the check is a simple binary search. We know that the base we're looking for is at least 3, at most $n - 1$, and the bigger the base we choose, the larger the value of the binary string will be. Find the threshold b where the binary string has value $\leq n$ in base b , but a value $> n$ in base $b + 1$. Then, check whether it has exactly the value n in base b , and if it does, we found a potential answer. Be careful about integer overflows when implementing this step.

For a simple optimization we can observe that larger binary strings correspond to smaller bases required to represent any number, and as we are looking for the smallest base that works, we can go through the binary strings from largest to smallest, and output the first one we find that works. E.g. notice that we will check the string 11 at the very end, outputting $b = n - 1$ if we didn't find any solution that would correspond to a longer binary string = a smaller base.

The time complexity is quite funny. Let's say your constant is some power of 2, so 2^x . The first part, trying all bases up to 2^x will take $O(2^x \log n)$. This is increasing as x gets larger. The second part, going through all the binary strings and doing a binary search, will take $O(2^{\frac{\log n}{x}} \log n)$. This is decreasing as x gets larger. Since we want to minimize their sum, let's find where they are equal. Doing some cancelling out, we arrive at $x = \sqrt{\log n}$.

So our time complexity is $O(2^{\sqrt{\log n}} \log n)$.

Of course, we could play fast & loose during the contest; any reasonable constant between about 100 to 10^5 should suffice for the limit $n = 10^{18}$.

Alternatively, given the nature of the contest, you can reach for some powerful factorization tools, and use the approach from the second subtask here as well. Numbers up to 10^{18} can be factorized in a matter of seconds, so you can solve the entire input file in the order of minutes using the slower algorithm. We could have given a way larger upper bound on n to prevent these solutions, but ultimately we decided to keep the task more approachable (given that this is just the first round) and solvable without having to deal with big integers.